Informe de Análisis de Resultados – Caso 3

Juan Manuel Forero Bohorquez / 202514666
Santiago Alvear Garzón / 202510185

1. Para cada pareja de objetos que interactúan ¿cómo se realiza la sincronización, así como el funcionamiento global del sistema?

R/ Todos los métodos de interacción entre parejas de objetos están protegidos con el término "synchronized" en su signatura. El sistema inicia los tres controladores (procesador, memoria y almacenamiento), además del buzón y la zona de espera. Los usuarios depositan solicitudes en el buzón de forma concurrente, y los gestores las reciben y "consumen" aplicando espera semi-activa, creando una máquina virtual por cada una. Cada MV intenta reservar procesador, memoria, y almacenamiento, pero si no lo consiguen, entran en espera pasiva. Cuando una MV finaliza, libera los recursos reservados y avisa/despierta a las otras MV para continuar con el ciclo. Si se alcanza el umbral, los gestores dejan de tramitar solicitudes hasta que la zona se libere de nuevo. El sistema acaba cuando todos los usuarios terminen de enviar solicitudes.

---------- ||| ----------

2. ¿Cómo implementó la espera "total" del gestor en el caso donde las solicitudes pendientes son >= umbral?

R/ Se implementó en dos partes: La primera es una verificación; el gestor verifica si la zona de espera alcanzó o superó el umbral, revisando si hay demasiadas máquinas virtuales "dormidas". Si efectivamente se superó el límite, continúa con la segunda parte, la cual consiste en bloquear al gestor y esperar por medio de un "wait()", de modo que ya no se reciben nuevas solicitudes hasta que se libere la zona de espera. Cuando se despeje **toda** la zona, se reanuda el procesamiento de solicitudes del gestor, y cada MV nueva lucha de nuevo por los recursos, y el ciclo se repite hasta terminar.

---------- ||| ----------

3. Una máquina reserva los recursos que necesita -uno a uno- y siempre en el orden establecido: procesador, memoria y disco. ¿Por qué es importante que todas sigan el mismo orden de solicitud?

R/ No es que exactamente esa secuencia se deba cumplir, sino que cada MV reserve recursos con un orden particular. De no hacerlo así, se puede incurrir en una espera circular propiciando la generación de un deadlock, o dicho de otro modo, si cada MV reserva y posee recursos distintos, luego ninguna podrá continuar, ya que estarán

esperando un recurso que está siendo retenido por otra MV, generando un bloqueo parcial o indefinido.

---------- ||| ----------

4. Si una máquina virtual, después de haber recibido uno o más recursos, encuentra que el siguiente recurso que necesita no está disponible, debe liberar inmediatamente todos los recursos que había reservado previamente. Por ejemplo, si una máquina ya recibió procesadores y memoria, pero no puede obtener el disco requerido, deberá devolver tanto los procesadores como la memoria. ¿por qué es importante liberar los recursos asignados cuando no se logra completar la reserva?

R/ Es necesario devolver o liberar los demás recursos ya que hay otras Máquinas Virtuales que también los necesitan, y retener los que posee indefinidamente hasta que se libere el que hace falta, implicaría una gran pérdida de tiempo que aumentaría tanto la complejidad temporal como la espacial. También habría inconsistencia en la actividad de los controladores, pues indicarían que ciertos recursos están ocupados, cuando en realidad están retenidos sin que la MV realmente los utilice.
En resumen, por cuestiones de optimización, la MV tiene dos opciones: obtiene los tres recursos, o no obtiene ninguno.

---------- ||| ----------

5. Cuando termine su ejecución, la máquina deberá liberar todos los recursos que tenía asignados y despertar otra máquina para que use los recursos. ¿debería despertar una sola o todas? ¿por qué?

R/ A todas, pues cada MV recibirá el aviso de que los recursos están disponibles, e intentará reservar de forma sincronizada compitiendo con todas las demás. Despertar a una sola máquina podría generar demoras en la ejecución, ya que se condicionaría a que solo cierta MV deba recibir los 3 recursos y hasta que no los obtenga, trabaje con ellos, y los libere, ninguna otra MV podrá continuar, y es precisamente lo que se busca evitar, tal como se justificó en la pregunta anterior.

SECCIÓN DE VALIDACIÓN
Aspectos Verificados:

- **Aspecto 1: Validación de parámetros**

Pruebas para verificar funcionamiento:

1. Ejecutar el programa sin indicar el archivo de configuración:
   Comando usado:

```
java -cp out Main
```

   Respuesta recibida:

```
Uso: java Main <archivo_configuracion>
```

   El programa finaliza de forma esperada, sin cortes abruptos y sin lanzar excepciones, sino con un mensaje indicativo del caso considerado.

2. Ejecutar el programa con un parámetro inválido:
   Parámetros
   NU = 0
   base = 1
   NG = 1
   tam1 = 1
   tam2 = 1
   procesadores = 2
   memoriaGiB = 1
   tamPaginaKB = 1048576
   almacenamientoGiB = 1
   tambloqueKiB = 1048576

   Repuesta recibida:

```
NU debe ser mayor que cero.
```

3. Ejecutar el programa con una capacidad incompatible con el tamaño de página o bloque:
   Parámetros
   NU = 1
   base = 1
   NG = 1
   tam1 = 1
   tam2 = 1
   procesadores = 1
   memoriaGiB = 1
   tamPaginaKB = 700000
   almacenamientoGiB = 1
   tambloqueKiB = 1048576

   Respuesta recibida:

```
La memoria debe ser múltiplo del tamaño de página.
```

---------- ||| ----------

- **Aspecto 2: Ejecuciones funcionales con parámetros permitidos**

Pruebas para verificas funcionamiento:

1. Ejecutar con un solo usuario y un solo gestor:
   Parámetros*
   NU = 1
   base = 1
   NG = 1
   tam1 = 1
   tam2 = 1
   procesadores = 2
   memoriaGiB = 1
   tamPaginaKB = 1048576
   almacenamientoGiB = 1
   tambloqueKiB = 1048576

   Respuesta recibida:

   ```
   Sistema terminado correctamente.
   ```

2. Ejecutar con tres usuarios y base = 1:
   Parámetros*
   NU = 3
   base = 1
   NG = 1
   tam1 = 1
   tam2 = 2
   procesadores = 4
   memoriaGiB = 4
   tamPaginaKB = 1048576
   almacenamientoGiB = 4
   tambloqueKiB = 1048576

   Respuesta Recibida:

   ```
   solicitudes depositadas = 6
   solicitudes retiradas = 6
   solicitudes canceladas = 0
   buzón vacío
   Sistema terminado correctamente.
   ```

---------- ||| ----------

- **Aspecto 3: Sincronización entre threads**

Pruebas para verificar funcionamiento:

1. Ejecutar con más solicitudes que recursos disponibles:
   Parámetros
   NU = 2
   base = 1
   NG = 2
   tam1 = 2
   tam2 = 1
   procesadores = 1
   memoriaGiB = 1
   tamPaginaKB = 1048576
   almacenamientoGiB = 1
   tambloqueKiB = 1048576

   Resultado Recibido:

   ```
   solicitudes depositadas = 3
   solicitudes retiradas = 3
   solicitudes canceladas = 0
   buzón vacío
   Sistema terminado correctamente.
   ```

2. Probar una configuración inválida de recursos:
   Parámetros
   NU = 1
   base = 1
   NG = 1
   tam1 = 1
   tam2 = 1
   procesadores = 1
   memoriaGiB = 1
   tamPaginaKB = 700000
   almacenamientoGiB = 1
   tambloqueKiB = 700000

   Resultado Recibido:

   ```
   La memoria debe ser múltiplo del tamaño de página.
   ```

   Aunque los bloques y el tamaño de página son compatibles, no dejan de ser incompatibles con el tamaño de memoria, y eso genera el mismo mensaje de error.

3. Verificar la liberación de recursos cuando se finaliza una ejecución válida:
   Parámetros
   NU = 2

base = 1
NG = 2
tam1 = 2
tam2 = 2
procesadores = 2
memoriaGiB = 2
tamPaginaKB = 1048576
almacenamientoGiB = 2
tambloqueKiB = 1048576

Respuesta recibida:

```
solicitudes depositadas = 3
solicitudes retiradas = 3
solicitudes canceladas = 0
buzón vacío
Sistema terminado correctamente.
```

Como se esperaba, efectivamente la ejecución funciona, y le liberan todos los recursos por parte de todas las MV's.